

Project – Part 1. Query Parsing & CLI Frontend

Головна ціль:

Зробити «вхід» у вашу СУБД: CLI та підсистему парсингу, яка:

- приймає SQL-запит у вигляді рядка,
- розбирає його на токени та будує **AST (Abstract Syntax Tree)**,
- перевіряє базову коректність запиту,
- видає:
 - або коректне AST,
 - або зрозумілу діагностику помилки.

На цьому етапі **ніякого реального виконання запиту ще не потрібно** — тільки парсинг і валідація.

2. Обсяг задачі та підтримувана підмножина SQL (або будь-якої іншої мови запитів, навіть вигаданої)

2.1. Обов'язкові типи запитів

Парсер **обов'язково** має підтримувати такі оператори:

1. **CREATE TABLE**

Мінімальний синтаксис:

```
CREATE TABLE table_name (  
  column_name1 INT,  
  column_name2 TEXT,
```

...
);

- Допускаються типи: **INT**, **BIGINT**, **TEXT** (можна розширити, але не обов'язково).
- Без primary key / foreign key / constraint-ів на цьому етапі (якщо є бажання можна як додаткову фічу).

2. INSERT INTO ... VALUES

Мінімальний синтаксис:

```
INSERT INTO table_name VALUES (1, 'Alice', 42);  
INSERT INTO table_name VALUES (1, 2, 3);
```

- Значення:
 - цілі числа (**123**, **-5**),
 - рядки в одинарних лапках (**'Alice'**),
 - **NULL** (опційно, але бажано).

3. SELECT

Мінімальний синтаксис:

```
SELECT col1, col2 FROM table_name;  
SELECT * FROM table_name;  
SELECT col1, col2 FROM table_name WHERE col1 > 10;  
SELECT col1 FROM table_name WHERE col2 = 'Alice' LIMIT 10;
```

- Обов'язкові можливості:
 - список колонок або *****,
 - **FROM** з однією таблицею,

- **WHERE** з простими предикатами:
 - порівняння `=`, `!=`, `<`, `<=`, `>`, `>=`,
 - логічні оператори **AND**, **OR**,
 - дужки в boolean-виразах (опційно, але бажано),
- **LIMIT n** (ціле число).
- Не потрібно:
 - **JOIN**,
 - **GROUP BY / HAVING**,
 - підзапити,
 - **CTE (WITH)**,
 - **UNION/INTERSECT/EXCEPT** (можна як розширення).

3. Варіанти реалізації

Ви обираєте один з двох підходів:

3.1. Варіант A: DIY Parser

- Ви **самостійно** реалізуєте:
 - лексер,
 - парсер для вищевказаного підмножини SQL,
 - побудову AST.
- Можна:
 - писати рекурсивний спуск руками,

- або використовувати генератор парсерів (Flex/Bison, ANTLR, PLY, Lark, Nom, і т.п.) і написати тільки граматику, токени та AST.

3.2. Варіант B: Library-based Parser

- Ви берете готову бібліотеку для парсингу SQL у вашій мові:
 - наприклад, `sqlparser-rs`, `sqlglot`, `sqlparse`, `JSqlParser` і т.д. (конкретний вибір — на вас).
- Обов'язково:
 - адаптувати дерево розбору бібліотеки до вашого внутрішнього AST (не тягнути всю чужу структуру всередину решти системи),
 - інкапсулювати залежність від бібліотеки всередині `parser`-модуля.

4. CLI та інтеграція

4.1. Бінарник / `entrypoint`

Повинен бути один основний CLI-інструмент, наприклад:

```
db-cli --query "SELECT id, name FROM users WHERE age > 18"
```

Обов'язкові вимоги:

1. Параметр `--query`

CLI приймає один SQL-запит як аргумент:

```
db-cli --query "CREATE TABLE t(a INT)"
db-cli --query "SELECT col1 FROM t"
db-cli --query "INSERT INTO t VALUES(1,2,3)"
```

- Можна підтримати читання запиту зі `stdin` / файлу як додаткову фічу.

2. Режим debug AST

Окремий прапорець, наприклад:

```
db-cli --debug-ast --query "SELECT a FROM t WHERE a > 10"
```

В цьому режимі програма:

- парсить запит,
- виводить **AST у зручному для читання форматі**: дерево, або JSON-подібна структура, або pretty-printed.

3. Режим перевірки синтаксису

- Може бути тим самим режимом, що й `--query`:
 - при успішному парсингу повернути код завершення `0`,
 - при помилці код `≠ 0` і повідомлення про помилку в `stderr`.

5. Вимоги до AST

AST — це ваша **внутрішня модель запиту**, яка буде використовуватися в наступних частинах проекту (планування, виконання, оптимізація).

5.1. Загальні вимоги

1. Типізовані вузли

- Для кожного виду запиту — окремий тип вузла/структури:
 - `AstNode::Select(SelectStmt)`,
 - `AstNode::CreateTable(CreateTableStmt)`,
 - `AstNode::Insert(InsertStmt)`.

- Для виразів:
 - Expr::ColumnRef,
 - Expr::LiteralInt,
 - Expr::LiteralString,
 - Expr::BinaryOp,
 - Expr::LogicalOp,
 - Expr::ComparisonOp,
 - Expr::UnaryOp (опційно).

2. Семантично осмислена структура

Наприклад, `SelectStmt` має поля:

```
SelectStmt {  
  projections: Vec<ProjectionItem>, // список колонок або *  
  from: TableRef, // одна таблиця  
  selection: Option<Expr>, // WHERE  
  limit: Option<u64>, // LIMIT  
}
```

CreateTableStmt:

```
CreateTableStmt {  
  table_name: String,  
  columns: Vec<ColumnDef>,  
}
```

InsertStmt:

```
InsertStmt {  
  table_name: String,  
  values: Vec<Expr>, // або Vec<Vec<Expr>> для багатьох рядків (якщо захочеш)  
}
```

3. Відокремленість від бібліотеки (для варіанту з library)

- AST жодним чином не залежить від зовнішнього API парсера.
- Адаптер робить перетворення:

■ `LibraryParseTree` » `YourAST`.

5.2. Pretty-print / debug-вивід

- Має бути **одна функція / метод**, який перетворює AST у людинозрозумілий текст.
- Важливо, щоб по виводу AST можна було перевірити:
 - які саме поля наповнені,
 - як оброблений `WHERE`,
 - що відбувається з `LIMIT`, `*`, іменами таблиць/колонок.

6. Лексер та парсер (для DIY варіанта)

Якщо обрано **DIY**:

6.1. Лексер

- Повинен виділяти токени для:
 - ключових слів: `SELECT`, `FROM`, `WHERE`, `CREATE`, `TABLE`, `INSERT`, `INTO`, `VALUES`, `LIMIT`, `AND`, `OR`, `NULL`,
 - ідентифікаторів (імена таблиць і колонок),
 - літералів:
 - цілі числа,
 - рядки в `'...'`,

- операторів: `=, !=, <, <=, >, >=`,
- роздільників: `(, ), ,, :, *`.
- Ігнорувати:
 - пробіли,
 - таби,
 - переноси рядків,
 - (опційно) SQL-коментарі `-- ...` або `/* ... */`.

6.2. Парсер

- Альтернативи:
 - рекурсивний спуск,
 - або генератор парсера (Bison/Yacc/PLY/ANTLR і т.д.).
- Має бути чітко сформована **граматика** (можна додати у README розділ “Grammar”).
- Важливо:
 - коректно задавати пріоритети й асоціативність для логічних/порівняльних операторів,
 - при неоднозначності — однозначно обирати трактування (через порядок правил, пріоритети, дужки).

7. Вимоги до діагностик і валідації

7.1. Синтаксичні помилки

Парсер повинен:

- **не падати** з exception'ом або segmentation fault при некоректному SQL,
- виводити зрозуміле повідомлення з:
 - текстом помилки,
 - позицією (рядок/колонка або хоча б індекс у рядку),
 - фрагментом проблемного місця (якщо можливо).

Приклади:

- Syntax error near 'SELEC' at position 1: did you mean 'SELECT'?
- Unexpected token 'FROM' at position 15, expected identifier.
- Unexpected end of input: expected ')' before end of query.

7.2. Базова структурна валідація

Навіть без знання каталогу/схеми, парсер повинен ловити речі типу:

- `SELECT FROM t` — немає списку колонок,
- `INSERT INTO t VALUES` — немає списку значень,
- `CREATE TABLE t()` — порожній список колонок (можна вважати помилкою),
- `WHERE` без виразу.

На цьому етапі **не потрібно**:

- перевіряти, що таблиця/колонки реально існують у каталозі,
- перевіряти типову сумісність (INT vs TEXT і т.д.).